

TUGAS LOAD BALANCING KOMPUTASI PARALEL Materi: MPI	Nama: Farhan Kamil Hermansyah NRP: 152024150 Mata Kuliah: IFB 206 - Komputasi Paralel dan Sistem Terdistribusi Dosen Pengampu: Lisa Kristiana PhD
---	--

Example Code:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank() # ID dari proses saat ini
size = comm.Get_size() # Total proses yang berjalan

data_mentah = None
if rank == 0:
    data_mentah = [i * 12.3 for i in range(size)]
    print(f"[Master - Proses {rank}] Data awal yang akan disebar: {data_mentah}\n")
    print("-" * 40)

data_lokal = comm.scatter(data_mentah, root=0)

hasil_lokal = data_lokal + 5
print(f"[Proses {rank}] Menerima data '{data_lokal}', setelah ditambah 5 hasilnya: {hasil_lokal}")

hasil_akhir = comm.gather(hasil_lokal, root=0)

if rank == 0:
    print("-" * 40)
    print(f"[Master - Proses {rank}] Hasil akhir setelah dikumpulkan dari semua proses: {hasil_akhir}\n")
```

Output:

```
D:\LIFE\ KULIAH ITENAS \SEMESTER 4\KOMPUTASI PARALLEL\LECTURE 8>mpiexec -n 4 python mpi.py
[Proses 1] Menerima data '12.3', setelah ditambah 5 hasilnya: 17.3
[Proses 3] Menerima data '36.900000000000006', setelah ditambah 5 hasilnya: 41.900000000000006
[Proses 2] Menerima data '24.6', setelah ditambah 5 hasilnya: 29.6

[Master - Proses 0] Data awal yang akan disebar: [0.0, 12.3, 24.6, 36.900000000000006]

-----
[Proses 0] Menerima data '0.0', setelah ditambah 5 hasilnya: 5.0
-----

[Master - Proses 0] Hasil akhir setelah dikumpulkan dari semua proses: [5.0, 17.3, 29.6, 41.900000000000006]
```

Analisis:

Proses dimulai dengan inisialisasi lingkungan global menggunakan objek `MPI.COMM_WORLD` untuk menetapkan identitas unik (rank) dan total proses (size) yang dialokasikan. Pada tahap awal ini, sistem menggunakan model memori terdistribusi di mana alokasi data bersifat terpusat pada proses Master (rank 0) yang secara eksklusif menyiapkan memori untuk variabel data mentah. Akibatnya, memori pada proses Worker lainnya tetap kosong dan tidak memiliki akses langsung ke data tersebut sebelum fase distribusi dimulai.

Mekanisme "berbagi" data dilakukan melalui fase distribusi atau *Scatter*, di mana array utama dipecah secara merata oleh Master menggunakan fungsi `comm.scatter`. Setiap potongan data kemudian dikirimkan dan dialokasikan ke dalam variabel lokal di memori masing-masing node. Hal ini memungkinkan setiap inti prosesor untuk menjalankan komputasi paralel secara independen dan simultan pada memori lokal mereka sendiri, tanpa perlu memperebutkan akses ke ruang memori yang sama.

Tahap akhir adalah fase agregasi atau *Gather* yang berfungsi sebagai langkah sinkronisasi untuk mengumpulkan kembali hasil komputasi. Seluruh jaringan proses diperintahkan untuk mengirimkan nilai yang telah diproses di memori lokal mereka kembali ke proses Master. Master kemudian menyusun kembali kepingan-kepingan paket informasi tersebut menjadi satu struktur data utuh yang terpusat, sehingga hasil akhir dari seluruh node dapat diakses kembali dalam satu variabel kolektif.

Simulasi Output Eksekusi

Jika skrip dieksekusi menggunakan utilitas peluncur sistem paralel dengan konfigurasi 4 proses (misalnya melalui `mpiexec -n 4`), matriks data akan terpetakan sebagai berikut:

-
- **Data Awal (Master):** [0.0, 12.3, 24.6, 36.9]
 - **Komputasi Proses 0 (Master):** Menerima input 0.0 → Modifikasi nilai → 5.0
 - **Komputasi Proses 1 (Worker):** Menerima input 12.3 → Modifikasi nilai → 17.3
 - **Komputasi Proses 2 (Worker):** Menerima input 24.6 → Modifikasi nilai → 29.6
 - **Komputasi Proses 3 (Worker):** Menerima input 36.9 → Modifikasi nilai → 41.9
 - **Data Agregasi Akhir (Master):** [5.0, 17.3, 29.6, 41.9]

```
D:\LIFE\ KULIAH ITENAS \SEMESTER 4\KOMPUTASI PARALLEL\LECTURE 8>mpiexec -n 4 python mpi.py
[Proses 1] Menerima data '12.3', setelah ditambah 5 hasilnya: 17.3
[Proses 3] Menerima data '36.900000000000006', setelah ditambah 5 hasilnya: 41.900000000000006
[Proses 2] Menerima data '24.6', setelah ditambah 5 hasilnya: 29.6

[Master - Proses 0] Data awal yang akan disebar: [0.0, 12.3, 24.6, 36.900000000000006]

-----
[Proses 0] Menerima data '0.0', setelah ditambah 5 hasilnya: 5.0
-----

[Master - Proses 0] Hasil akhir setelah dikumpulkan dari semua proses: [5.0, 17.3, 29.6, 41.900000000000006]
```

Kesimpulan

Skrip uji secara empiris memvalidasi konsep dasar pengiriman pesan (*message passing*) pada kluster komputasi. Operasi kolektif scatter dan gather terbukti memberikan metode manajemen beban kerja yang terstruktur dan jauh lebih mangkus dibandingkan dengan mendefinisikan rute paket komunikasi *point-to-point* secara manual.
